

EMPOWERING THE DEVELOPER

Chapter 6

Servlet Session Tracking



Objectives

- **How to do session tracking:**
 - Demonstrate why session tracking is important.
 - Discuss and show some traditional session tracking techniques.
 - Illustrate servlet session APIs .
 - How to implement session tracking.
- **Lab exercise:**
 - Write a servlet for tracking the personal information.

How Do We Need HTTP State?

- Web applications need to track the users across a series of requests:
 - Online shopping (e.g. Order books).
 - Financial portfolio manager.
 - Movie listings.
- HTTP does not support directly.
- Need a mechanism to maintain **state** about a series of requests from the same user (or originating from the same browser) over some period of time.

User Authentication

- Use authentication to track a user through a site:
 - The user has to login when visiting the site.
 - Each request includes the login information such as the user's name.
- How to support HTTP Authentication?
 - Set up with admin tool (e.g. Tomcat web.xml).
 - The server and browser take care of the detail.
- How to get the user's name in a servlet?

```
String userName = request.getRemoteUser();  
String[] cartItems = getItemsFromCart(userName);
```

Discussion: Authentication

- **Advantages:**

- Easy to implement.
- The user may access the site from different machine.
- Sessions may last indefinitely.

- **Disadvantages:**

- User needs to create an account on the server.
- Requires a login process the first access.
- Not easy to support anonymous users.

URL Rewriting

- URLs can be rewritten or encoded to include session information.
- URL rewriting usually includes a **session id**.
- id can be sent as extra path information:
 - <http://.../servlet/Rewritten/688>
 - Works well if no need for extra path info.
- id can be sent as an added parameter:
 - <http://.../servlet/Rewritten?sessionId=688>
 - Doesn't work with POST, cause name clash.
- Id can be sent by a custom change technique:
 - [http://.../servlet/Rewritten;\\$sessionId\\$688](http://.../servlet/Rewritten;$sessionId$688)
 - May not work for all servers.

Discussion: URL Rewriting

- **Advantages:**
 - Let user remain anonymous.
 - They are universally supported (most styles).
- **Disadvantages:**
 - Tedious to rewrite all URLs.
 - Only works for dynamically created documents.

Hidden Form Fields

- Hidden form fields are another way to support session tracking.
- Hidden form fields do not display in the browser, but can be sent back to the server by submit.

```
<FORM ACTION="/servlet/ShowParameters" METHOD="POST">  
...  
<INPUT TYPE="HIDDEN" NAME="OCCUPATION" VALUE="ENGINEER">  
<INPUT TYPE="HIDDEN" NAME="SESSIONID" VALUE="194043">  
...  
.  
..  
</FORM>
```

- Fields can have identification (session id) or just some thing to remember (occupation).
- Servlet reads the fields using `req.getParameter()`.

Discussion: Hidden Form Fields

- **Advantages:**
 - Universally supported.
 - Allow anonymous users
- **Disadvantages:**
 - Only works for a sequence of dynamically generated forms.
 - Breaks down with static documents, emailed documents, bookmarked documents.
 - No browser shutdowns.

Using Cookies in Servlets

- **Cookie definition:**
 - Web server sends a cookie name and value to a browser and later can read them back from the browser.
- **The process:**
 - Servlet sends a cookie with its **response** to the client .
 - The client saves the cookie.
 - The client returns a cookie back with subsequent **requests** (depends on some rules).
- **Typical Uses of Cookies**
 - Identifying a user during an e- commerce session.
- **Cookies can save either information or**

Programming Cookies

- Servlet API supports cookies:
 - `javax.servlet.http.cookie`
- `Response.addCookie(Cookie)` add cookies to a response.

```
Cookie cookie = new Cookie("name", "value");  
response.addCookie(cookie);
```

- `request.getCookie()` get cookie from a request.

```
Cookie[] cookie =request.getCookie();  
if (cookie != null) {  
    for (int i= 0; i< cookie.length; i++) {  
        String name = cookie[i].getName();  
        String value = cookie[i].getValue();  
    }  
}
```

Cookie Methods

- **GetDomain/ setDomain**
 - Lets you specify domain to which cookie applies. Current host must be part of domain specified.
- **getMaxAge/ setMaxAge**
 - Gets/ sets the cookie expiration time (in seconds). 0 seconds is a delete command, and negative value expires at browser shutdown (default). If you fail to set this, cookie applies to current browsing session only.
- **getName/ setName**
 - Gets/ sets the cookie name. For new cookies, you supply name to constructor, not to setName. For incoming cookie array, you use getName to find the cookie of interest.

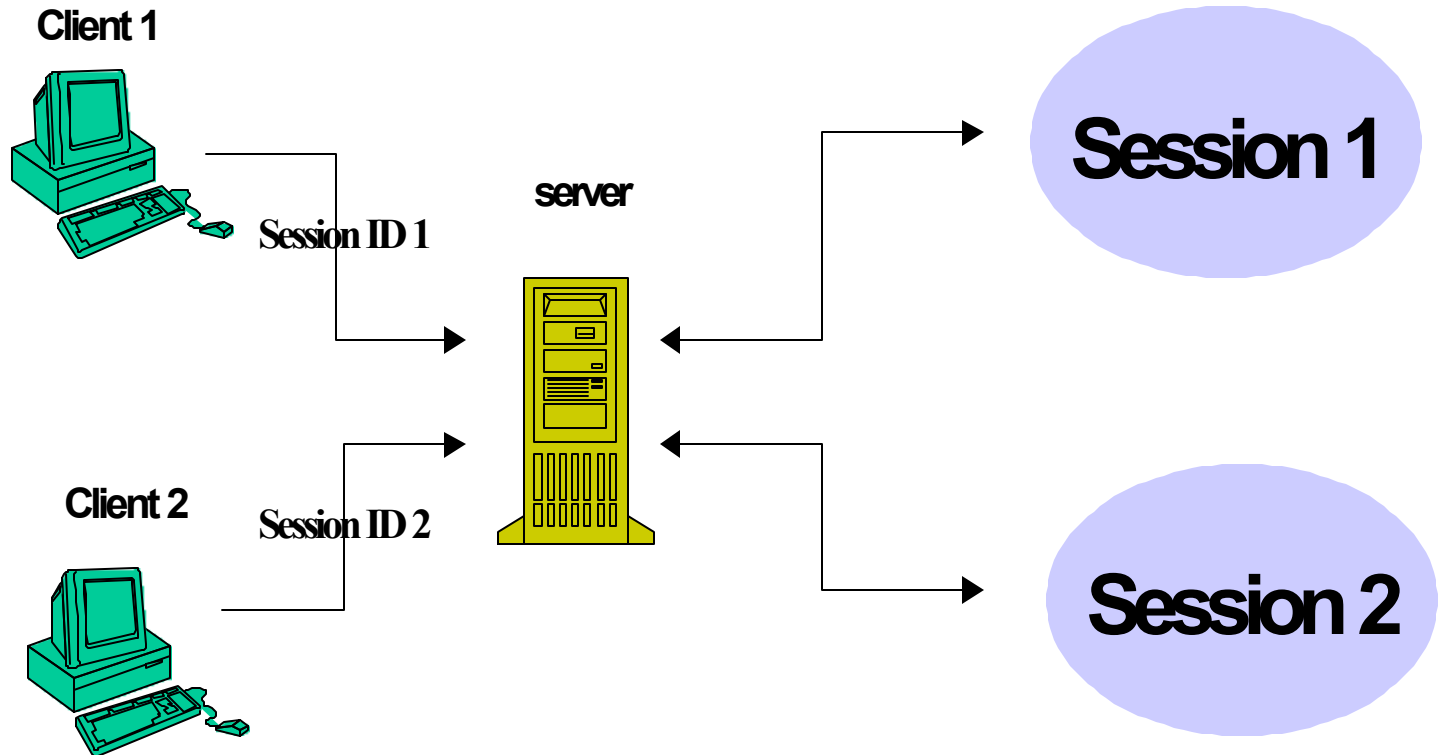
Cookie Methods (Cont.)

- **getPath/ setPath**
 - Gets/ sets the path to which cookie applies. If unspecified, cookie applies to URLs that are within or below directory containing current page.
- **getSecure/ setSecure**
 - Gets/ sets flag indicating whether cookie should apply only to SSL connections
- **getValue/ setValue**
 - Gets/ sets value associated with cookie. For new cookies, supply value to constructor, not to **setValue**. For incoming cookie array, use **getName** to find cookie of interest, then call **getValue** on result.
- **setVersion**
 - Sets cookie format to use. 0 is netscape, 1 is still experimental RFC 2109

Discussion: Cookies

- **Advantages:**
 - Very easy to implement.
 - Highly customizable.
 - Persist across browser shut-downs.
- **Disadvantages:**
 - Often: users turn off cookies for privacy or security reason.
 - Not quite universal browser support.

Sessions



Session Tracking Overview

- The servlet API has a built-in support for session tracking.
- Session objects live on the server.
 - Each user has associated an `HttpSession` object-- one user/session.
 - It operates like a `hashtable`.
- To get a user's existing or new session object:
 - `HttpSession session = request.getSession(true);`
 - "`true`" means the server should create a new session object if necessary.
- To store or retrieve an object in the session:
 - stores values: `setAttribute("cartItem", cart);`
 - retrieves values: `getAttribute("cartItem");`

Session Tracking API

- **getAttribute [2.2]**
 - retrieves a previously stored value from a session. Returns null if no value found.
- **setAttribute [2.2]**
 - Stores a value in a session.
- **removeAttribute [2.2]**
 - Removes values associated with name.
- **String[] session.getAttributeNames [2.2]**
 - Returns names of all attributes in the session.
- **getId**
 - Returns the unique identifier.

Session Lifecycle API

- Sessions usually timeout after 30 minutes of inactivity.
 - A different timeout may be set by server admin.
- **public void invalidate()**
 - Expires the session and unbinds all objects with it.
- **boolean session.isNew()**
 - Determines if session is new to client (not page).
- **long session.getCreationTime()**
 - Returns time at which session was first created.
- **long session.getLastAccessedTime()**
 - Returns when the user last accessed the server.
- **getMaxInactiveInterval, setMaxInactiveInterval**
 - Gets or sets the amount of time, session should go without access before being invalidated

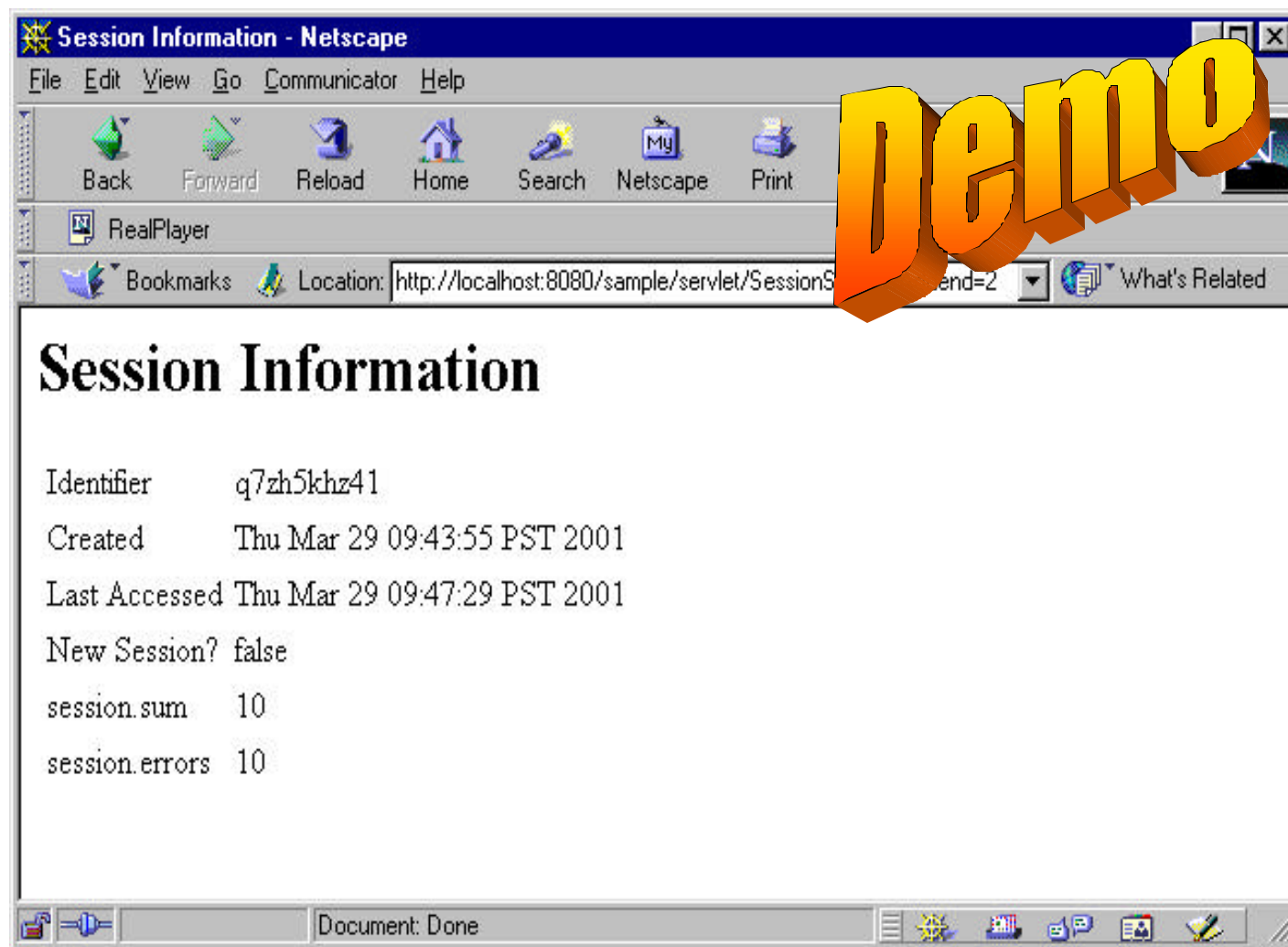
Session Tracking

```
public class SessionServlet extends HttpServlet {
    private static final String SUM_KEY = "session.sum";
    private static final String ERROR_KEY = "session.errors";
    public void doGet (HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {
        HttpSession session = request.getSession(true);
        Integer sum = (Integer)session.getAttribute(SUM_KEY);
        int ival = 0;
        if (sum != null) {
            ival = sum.intValue();
        }
        try {
            String addendString = request.getParameter("Addend");
            int addend = Integer.parseInt (addendString);
            sum = new Integer(ival + addend);
            session.setAttribute(SUM_KEY, sum);
        } catch (NumberFormatException e) {
            ...
        }
        //continue on next slide.
    }
}
```

Session Lifecycle Management

```
out.println("<td>" + session.getId() + "</td></tr>");
out.println("<tr><td>Created</td>");
out.println("<td>"
+ new Date(session.getCreationTime()) + "</td></tr>");
out.println("<tr><td>Last Accessed</td>");
out.println("<td>"
+ new Date(session.getLastAccessedTime())
+"</td></tr>");
out.println("<tr><td>New Session?</td>");
out.println("<td>" + session.isNew() + "</td></tr>");
Enumeration names= session.getAttributeNames();
String name = null;
while (names.hasMoreElements()) {
    name =(String)names.nextElement();
    out.println("<tr><td>" + name + "</td>");
    out.println("<td>" + session.getAttribute(name) +
"</td></tr>");
}
```

Show Session Demo



Session Tracking Usage

- When clients at an on- line store add an item to their shopping cart, how does the server know what's already in the cart?
- When clients decide to proceed to checkout, how can the server determine which previously created shopping cart is theirs?

Obtain a Session

```
public class CatalogServlet extends HttpServlet {  
    public void doGet (HttpServletRequest request,  
                      HttpServletResponse response)  
        throws ServletException, IOException {  
        // Get the user's session and shopping cart  
        HttpSession session = request.getSession(true);  
        ...  
        out = response.getWriter();  
        ...  
    }  
}
```

Storing and Getting Data from a Session

```
public class CatalogServlet extends HttpServlet {
    public void doGet (HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {
        // Get the user's session and shopping cart
        HttpSession session = request.getSession(true);
        ShoppingCart cart =
            (ShoppingCart)session.getAttribute(
                "examples.bookstore.cart");
        // If the user has no cart, create a new one
        if (cart == null) {
            cart = new ShoppingCart();
            session.setAttribute("examples.bookstore.cart",
                                cart);
        }
        ... //see next slide.
    }
}
```


Storing and Getting Data from a Session (Cont.)

```
//If the user wants to add a book, add it and print  
the result  
String bookToAdd = request.getParameter("Add");  
if(bookToAdd != null) {  
    BookDetails book =  
        database.getBookDetails(bookToAdd);  
    cart.add(bookToAdd, book);  
    out.println("<p><h3>" + ...);
```

Invalidate the Session

```
public class ReceiptServlet extends HttpServlet {
    public void doPost(HttpServletRequest request,
                       HttpServletResponse response)
        throws ServletException, IOException {
        ...
        scart = (ShoppingCart)
            session.getAttribute("examples.bookstore.cart");
        ...
        // Clear out shopping cart by invalidating the session
        session.invalidate();

        // set content type header before accessing the Writer
        response.setContentType("text/html");
        out = response.getWriter();
        ...
    }
}
```

Summary

- **Showed various techniques for session tracking:**
 - User authentication.
 - Hidden form fields.
 - URL rewriting.
 - Cookies.
 - The HttpSession class.
- **No technique is best for every situation.**
- **Choose the technique what's right for your application.**